

What the AI costs per CV, and how to cut it

An answer to one question: a single model call, one CV against one job description, should cost a few cents — so where does fifty cents a CV go? Written 16 September 2026, from reading the code. Nothing here has been measured yet: the local database holds no telemetry and the local environment has no API key, so Phase 0 of the plan replaces these estimates with real numbers.

1. Corrections to the plain-English summary

The shape of the summary is right. Two of its numbers need changing, one of its suspects is a dud, and two cost drivers were on neither list.

Claim	Verdict	What to say instead
The three fixes (two listed)	Missing the biggest one	Caching plus a shorter answer, still on Opus at high effort, lands at ~\$0.07–0.12 per CV — not 4¢. Thinking is most of the written output and neither fix touches it. ~4¢ needs effort high !' medium; ~2¢ needs Sonnet 5. Both are quality-gated, so they follow an eval.
"re-read them at a tenth of the price"	Right for reads, incomplete	Writing the cache costs 1.25× on the first call, and the entry only exists once that first response has begun. With four workers in parallel, all four first calls pay the write price unless one is sent ahead. The difference between "most" and "some" of the reading cost.
"can't cost more than about 25 cents"	Right	8,000 written tokens = \$0.20, plus ~\$0.06 read "H \$0.27 hard ceiling per CV.
"the AI being asked twice when the first answer fails"	Weak suspect	The SDK retries only on network, 429 and 5xx errors, none of which are billed. Triage batch retries (max 3, triage.js:60) redo only applicants not yet scored. A refusal is billed but is not retried — it falls back to the deterministic score. Worst case one wasted call, not two.
"testing"	Not a suspect	The local .env has no ANTHROPIC_API_KEY and the AI suites stub fetch. Local tests cost nothing.
"the free demo"	Real, and larger than it looks	The public demo runs the full Opus pipeline for anonymous visitors. See section 2.
"CVs uploaded as pictures"	Real, small	One vision call per image at low effort, about 3–5¢ a page.
"your database records the real numbers"	True for Triage only	Search records no tokens at all, and demo spend is invisible. Once caching lands, the recorded input_tokens will undercount unless the cache fields are stored too.
"the later explanation must be given the verdicts, not asked to decide again"	Right, and important	Keep it. One modification: most of that explanation needs no model at all — strengths and gaps are derivable from the verdicts in code, for nothing.

2. Two things the code says that change the picture

A launched Triage analyses fifty CVs, not twenty-five

TRIALGE_INITIAL_DEEP defaults to 50 (server/src/triage.js:46); 25 is the rolling tranche fetched afterwards as the recruiter scrolls. So a Triage commits to roughly **\$6–9** the moment it is launched, before anyone has scrolled anywhere, and about **\$12–18** for a 100-CV pile read to the end. The "\$12.50 per 100 free CVs" in the summary

is about right; the earlier "first 25" framing was not.

The public demo runs the full Opus pipeline for anonymous visitors

runPublicSearch (server/src/publicDemo.js:198) calls the same runSearch a paying recruiter does: the JD is parsed, twenty-five candidates are analysed at high effort, and six are shown. Identical job descriptions share a job row — 144 demo searches have so far produced only 7 jobs — but every distinct JD a stranger pastes costs about **\$3–5**. The only limit is eight searches per hour per browser fingerprint: no global ceiling, no daily allowance, and no telemetry. It is the one uncapped surface in the product, and the first place to look if the real bill is materially above fifteen cents a CV.

Two smaller ones

- **Nothing is cancelled when a recruiter closes the tab.** No abort signal is wired from the HTTP request into runSearch, so a page of twenty-five analyses runs to completion whether or not anyone is waiting.
- **Changing the model re-analyses everything once.** The model name is part of the analysis cache key (server/src/matching/analysis.js:24), as is VERSIONS.scoring. Handled in Phase 4 by reading rows written by any model while writing new ones under the current one.

3. Where a CV's money goes today

One CV against one JD is one call to analyseMatch (server/src/ai.js:1270), from either Search (matching/analysis.js:175) or Triage (triageQueue.js:623). Opus 5 is \$5 per million tokens read and \$25 per million written; the model's private thinking is billed as writing. Token counts use the current tokenizer (~2.5 characters per token), which makes them about 40% higher than my first estimate.

Part of the call	Tokens	Cost	Cacheable?
Instructions (MATCH_SYSTEM, 8.5k characters)	~3,400	\$0.017	yes
The answer schema (4.5k characters)	~1,800	\$0.009	yes
JD, requirement list, recruiter instruction	~2,000	\$0.010	yes, per job
The CV and dossier	~3,000–5,000	\$0.015–0.025	no
Read, total	~10,000–12,000	~\$0.05–0.06	~60% fixed
Thinking (adaptive, effort high)	~2,000–4,000	\$0.05–0.10	—
The JSON answer (verdicts, reasoning, strengths, gaps, transferable, evidence, probes, notes)	~1,500–2,000	\$0.04–0.05	—
Written, total	~3,500–6,000	~\$0.09–0.15	—
Per CV		~\$0.14–0.20	

Roughly 35% reading, 65% writing. max_tokens is 8,000 (ai.js:1310), so one CV cannot cost more than about \$0.27 however badly it goes. Caching addresses the 35%; a shorter answer addresses about a third of the 65%; thinking is the rest. Only the per-requirement verdicts decide the score — it is computed in code from them — and since the Triage card stopped showing reasoning, the rest is prose most recruiters never open.

What each fix is worth, cumulatively

Configuration	"H \$/CV	100 free CVs, read to the end
Today: Opus, high effort, full answer, no caching	0.15	\$15
+ prompt caching (Phase 1)	0.11	\$11

+ shorter answer (Phase 2)	0.08	\$8
+ effort medium (Phase 4a) — the summary's "4 cents"	0.04–0.05	\$4–5
Sonnet 5, medium, cached, short answer (Phase 4b)	0.02	\$2

The first two are quality-neutral and can ship together. The last two change what the model does and must not ship on anyone's intuition, mine included — they are gated on the eval in Phase 4.

4. Execution plan

Phase 0 — Measure. Half a day, nothing changes

- `npm run ai:cost: read triage_cost_events and ai_failures`, print dollars per CV by stage and model against a price table. Run it on Render, where the real rows are.
- Count the demo's spend to date: distinct `job_id` in `public_searches`, times about \$4.
- One free `countTokens` call on a real JD and CV to pin the true prefix size.

Exit condition: a measured cost per CV and a real read/write split. If it comes back at thirty cents or more, stop and find out why before changing anything — it would mean something is happening that reading the code does not explain.

Phase 1 — Savings with no effect on quality. One day

1. **Cache the shared prefix** in `analyseMatch`: mark the system block, and split the user message into [JD + criteria + instruction] and [this candidate]. Two of the four available breakpoints. The prefix is already in the right order, so nothing needs rearranging. Opus 5's minimum cacheable prefix is 512 tokens and ours is about 7,000; that caching coexists with a JSON schema needs proving on the first call rather than assuming.
2. **Warm the cache before fanning out**: send the first candidate's call, release the other three workers when it starts responding. No extra call, about two seconds of delay.
3. **Telemetry that survives caching**: record `cache_creation_input_tokens` and `cache_read_input_tokens` as well as the plain counts, and extend the same telemetry to Search and the demo, which record nothing today. Without this, Phase 1 makes the recorded token count look like a saving it is not.
4. **Cap the demo**: analyse only the eight candidates it might show rather than twenty-five, and add a global daily allowance beyond which the demo falls back to deterministic scoring and says so.
5. **Cancel abandoned searches**: wire an abort from the closed request into `runSearch`.

Acceptance: cache reads are at least half of everything read on the second and subsequent CVs of a page; the demo is capped; analysis-depth-check, ai-observability-check and the Triage suites pass.

Phase 2 — Ask for a shorter answer. One day

Keep in the main call: the per-requirement verdicts (status, quote, and a reason of at most twenty-five words), confidence, location fit, seniority alignment, and a two-sentence reasoning — the Search result card prints that line (`client/src/pages/HrPanel.jsx:5836`), so it has to be there.

Remove: evidence (it repeats the quotes already in the verdicts), strengths and gaps (derivable from the verdicts by tier, in code, for nothing), and move transferable and probes to Phase 3. Trim the instructions to match, which shrinks the cached prefix as well. The client reads derived strengths and gaps instead; both dialogs already null-check, so stored analyses in the old shape keep rendering. **VERSIONS.scoring must not be bumped** — scoring is unchanged, and bumping it would re-analyse every reopened search.

Acceptance: written tokens down at least 30%; the verdict distribution on the fixture set is unchanged (score-fairness-check).

Phase 3 — Write the explanation when someone opens the candidate. Half to one day

A new endpoint returns the stored explanation if one exists, and otherwise generates it: Sonnet 5, low effort, no thinking, given the verdicts that were already decided and told to explain them rather than to judge again. That

detail is the whole risk of this phase — an explanation that re-decides will eventually contradict the score on screen, and recruiters will see it. A code-side check discards any explanation that treats a requirement with no evidence as met.

About one to two cents per candidate a recruiter actually opens, against fifteen for every candidate whether opened or not. Most of the dialog — verdicts, derived strengths and gaps — renders instantly from stored data; only the written paragraph waits.

Phase 4 — Effort and model, decided by an eval. One to two days

This is the largest saving and the only part with real quality risk, so it is decided by measurement rather than by argument. `npm run ai:eval` runs about five job descriptions against about fifteen CVs under four configurations — Opus/high as the baseline, Opus/medium, Sonnet 5/high, Sonnet 5/medium — and reports, against the baseline: per-requirement verdict agreement (exact, and counting meets/partial as adjacent), the rank correlation of fits within each job, cost per CV, and latency. One run costs about \$25–30.

Proposed rule: adopt the cheapest configuration that agrees with the baseline on at least 90% of must-have verdicts and holds a rank correlation of 0.9 or better. Expectation, to be proved right or wrong: Opus at medium effort passes comfortably; Sonnet 5 at medium passes for Triage.

Model and effort become environment variables per call type, so the switch is a Render setting rather than a deploy, and the analysis cache reads rows written by any model while writing new ones under the current one — otherwise the switch silently re-analyses every open search.

Phase 5 — Guardrails. Half a day

- In the Console: a monthly spend limit, and a dedicated workspace for the production key so the limit is scoped to Cursus rather than to the whole account. Ten minutes, and not something code can do.
- In the app: a per-company daily ceiling on analyses, with the deterministic fallback and an honest notice; an `AI_PAUSED` switch; and `ai:health` naming the two spend-limit failures explicitly (HTTP 400 "usage limits", HTTP 429 `enforced_spend_limit_reached`) so a capped account is not mistaken for a broken one. Today a hit limit degrades results quietly, which is the wrong failure for something recruiters pay for.

Deliberately not doing: the Batch API for Triage

It is half price, but the first fifty CVs are what the recruiter is waiting for and the rest are fetched by scrolling. Both are interactive. Pre-analysing the whole pile in a batch buys a discount on CVs nobody scrolls to, which is not a saving.

5. Open decisions

1. **Triage initial depth:** keep 50, or drop to 25 with the next 25 queued immediately behind it? It only saves money when a recruiter abandons a Triage after the first screen. Recommendation: 25.
2. **Put the public demo on Sonnet 5 regardless of the eval?** It is a taster, and it is the one uncapped surface. Recommendation: yes.
3. **Eval material:** fixtures only, or three to five real anonymised job descriptions? Real ones make the decision rule mean something. Nothing would be taken from users' data.
4. **Sequencing:** Phases 1–3 in order, or start Phase 4's eval alongside Phase 2 so the model decision is ready when the rest lands?

About four to five working days in total. Fifteen cents to four or five is Phases 0–3 plus the effort change — roughly two and a half days. Two cents needs the Sonnet eval to pass.